



PROJETO DESENVOLVIMENTO WEB II

1. Apresentação

Os alunos deverão desenvolver, em grupo de 4 a 6 pessoas (não deve ser feito em grupos com número de alunos diferente do que mencionado acima), um projeto web criado integralmente do zero. O projeto será desenvolvido em duas partes:

- **Parte 1:** criação de uma interface web funcional utilizando HTML, CSS e JavaScript no navegador;
- **Parte 2:** evolução do projeto para uma aplicação integrada a um back-end, banco de dados e serviço web, com consumo de API e apresentação de dados dinâmicos.

A Parte 1 deverá demonstrar os conteúdos de HTML, CSS e JavaScript estudados até DOM, validação, eventos e módulos ES6. A Parte 2 deverá incorporar Node.js, Express, API REST, CRUD, persistência de dados, consumo de APIs, fetch, Promises e async/await.

2. Objetivos

Ao final do projeto, o aluno deverá demonstrar capacidade de:

- Estruturar páginas web com HTML semântico;
- Aplicar estilos com CSS externo;
- Criar interfaces organizadas e responsivas;
- Utilizar JavaScript no lado cliente;
- Trabalhar com variáveis, constantes, operadores e tipos de dados;
- Utilizar estruturas condicionais e laços de repetição;
- Criar e utilizar funções;
- Responder a eventos;
- Manipular elementos do DOM;
- Validar dados de formulários;
- Organizar código utilizando módulos ES6;
- Desenvolver uma API REST;
- Implementar operações de CRUD;
- Utilizar rotas, controladores e serviços;
- Armazenar e consultar dados;



- Conectar a aplicação a um banco de dados;
- Consumir uma API utilizando fetch, Promises e async/await;
- Construir uma página dinâmica com dados provenientes de um serviço web.

3. Definição do tema

Cada grupo deverá escolher um problema ou situação que possa ser apoiado por uma aplicação web.

Temas:

- Sistema de cadastro de livros;
- Catálogo de filmes ou séries;
- Agenda de consultas;
- Sistema de controle de tarefas;
- Cadastro de eventos;
- Controle de produtos;
- Sistema de receitas culinárias;
- Catálogo de jogos;
- Sistema de reservas;
- Plataforma de cursos;
- Controle de atividades acadêmicas;
- Sistema de doação de itens;
- Catálogo de animais para adoção.

O projeto deverá possuir:

- Pelo menos uma entidade principal;
- Dados que possam ser cadastrados, consultados, alterados e excluídos;
- Interface para interação com o usuário;
- Possibilidade de evolução para uma aplicação cliente-servidor.

Exemplo:

Tema: sistema de controle de livros.

Entidade principal: livro.

Dados possíveis: título, autor, gênero, ano de publicação, status de leitura e avaliação.



4. Parte 1 — Interface e JavaScript no navegador

Prazo: 31 de agosto de 2026 até 13 de setembro.

A entrega poderá ser realizada nas datas previstas no cronograma da disciplina, incluindo as aulas de entrega e revisão.

Objetivo

Na Parte 1, o grupo deverá criar a primeira versão do projeto, demonstrando os conhecimentos de HTML, CSS e JavaScript estudados até DOM, validação de campos, eventos e módulos ES6.

A aplicação ainda não precisará possuir servidor, banco de dados ou API.

4.1 Planejamento

Entregar um documento curto contendo:

- Nome do projeto;
- Tema escolhido;
- Problema que será resolvido (Ex: Disponibilizar livros para que sejam usados por alunos);
- Público-alvo;
- Objetivo da aplicação;
- Descrição das principais funcionalidades;
- Integrantes do grupo;
- Divisão inicial das tarefas.

4.2 Estrutura HTML

A interface deverá:

- Utilizar HTML5;
- Possuir estrutura semântica;
- Conter cabeçalho, conteúdo principal e rodapé;
- Apresentar títulos e textos explicativos;



- Possuir formulário relacionado ao tema;
- Possuir área para exibição dos dados;
- Utilizar campos adequados para cada informação;
- Utilizar label associado aos campos;
- Utilizar elementos apropriados, como form, input, select, textarea, button, table, lista ou cartões.

4.3 Estilização CSS

A interface deverá:

- Utilizar arquivo CSS externo;
- Apresentar identidade visual coerente;
- Utilizar classes e identificadores;
- Possuir espaçamentos adequados;
- Utilizar cores e tipografia legíveis;
- Estilizar formulários, botões e mensagens;
- Utilizar Flexbox, Grid ou outra técnica de organização;
- Possuir comportamento responsivo;
- Apresentar efeito visual de hover;
- Ser utilizável em computador e celular.

4.4 JavaScript básico

O código deverá demonstrar o uso de:

- let;
- const;
- Strings;
- Números;
- Booleanos;
- Arrays;
- Objetos;
- Operadores aritméticos;
- Operadores relacionais;
- Operadores lógicos;
- Comentários;
- console.log().



4.5 Estruturas de controle

O projeto deverá utilizar:

- if;
- else if;
- else;
- Pelo menos um laço for, while ou for...of;
- Pelo menos uma função com parâmetros;
- Pelo menos uma função com retorno;
- Uma função anônima ou arrow function.

4.6 Eventos

A interface deverá responder a ações do usuário, como:

- Envio de formulário;
- Clique em botão;
- Alteração de campo;
- Remoção de registro;
- Limpeza de dados.
- Etc...

Utilize **addEventListener()** para registrar os eventos.

4.7 Manipulação do DOM

O projeto deverá:

- Selecionar elementos da página;
- Ler valores dos campos;
- Alterar textos;
- Alterar classes ou atributos;
- Criar elementos dinamicamente;
- Remover elementos;
- Atualizar a interface após uma ação do usuário.



4.8 Validação

O formulário deverá validar os dados antes de processá-los. A validação deverá verificar, de acordo com o tema:

- Campos obrigatórios;
- Tamanho mínimo de textos;
- Valores numéricos válidos;
- Intervalos permitidos;
- Seleção de opções;
- Formatos específicos, quando necessário.

As mensagens de erro deverão ser claras e aparecer próximas ao formulário ou em uma área de mensagens.

4.9 Módulos ES6

O grupo deverá organizar parte do código utilizando módulos.

Estrutura sugerida:

```
parte-1/
├── index.html
├── css/
│   └── style.css
└── js/
    ├── main.js
    ├── validacao.js
    └── dados.js
```

Exemplo no HTML:

```
<script type="module" src="js/main.js"></script>
```

Exemplo de exportação:

```
export function validarTexto(texto) {
  return texto.trim().length > 0;
}
```



Exemplo de importação:

```
import { validarTexto } from "../validacao.js";
```

4.10 Funcionalidades mínimas

A aplicação deverá permitir, no mínimo:

- Inserir um registro;
- Validar os dados;
- Exibir o registro na tela;
- Editar ou remover um registro;
- Limpar os dados exibidos;
- Apresentar uma mensagem de sucesso ou erro;
- Exibir pelo menos um resumo ou indicador calculado.
- Arquivo local, para salvar valores da entidade;

As funcionalidades deverão funcionar durante a demonstração.

4.11 Entrega da Parte 1

A entrega deverá conter:

```
projeto_nome_grupo/  
├── README.md  
├── planejamento.pdf  
├── index.html  
├── css/  
│   └── style.css  
└── js/  
    ├── main.js  
    ├── validacao.js  
    └── dados.js
```

O README.md deverá conter:

- Nome do projeto;
- Integrantes;
- Tema;
- Como executar;



- Funcionalidades implementadas;
- Tecnologias utilizadas;
- Limitações conhecidas.

4.12 Avaliação da Parte 1

- Planejamento e definição do problema: 1,0 ponto;
- Estrutura HTML semântica: 1,0 ponto;
- Qualidade e organização do CSS: 1,0 ponto;
- Responsividade e usabilidade: 0,5 ponto;
- Variáveis, constantes, tipos e operadores: 0,5 ponto;
- Condicionais e laços: 1 ponto;
- Funções e organização do código: 1 ponto;
- Eventos e manipulação do DOM: 1,0 ponto;
- Validação de campos: 1 ponto;
- Uso de módulos ES6: 1 ponto;
- Funcionalidades: 1,0 ponto.

Totalizando 10 pontos, sujeito ao peso definido para a Parte 1 na avaliação da disciplina (disponível em plano de Ensino no moodle).

Cálculo da nota final

- **Cópias de parte ou todo no código terão nota zero.**
- Códigos feitos por IA's ou assistidos por IA's sobre os quais o grupo não saiba explicar terão nota igual a zero.
- **A nota final do projeto será dada pela fórmula:**
 - $0.5 \cdot NA + 0.25 \cdot NR + 0.25 \cdot NC$
 - onde NA denota a nota de apresentação, NR denota a nota do relatório enviado, e NC denota a nota de implementação/código-fonte.

5. Parte 2 — Aplicação cliente-servidor

Período de desenvolvimento

De setembro a 18 de novembro de 2026.

A Parte 2 deverá evoluir a aplicação criada pelo grupo. Não se trata de iniciar um projeto diferente, mas de transformar a interface da Parte 1 em uma aplicação com back-end, API e



persistência de dados.

5.1 Back-end

O grupo deverá criar um servidor utilizando:

- Node.js;
- Express;
- Rotas;
- Controladores;
- Serviços;
- Organização modular do código.

Estrutura sugerida:

```
parte-2/
├── frontend/
│   ├── index.html
│   ├── css/
│   │   └── style.css
│   └── js/
│       └── main.js
└── backend/
    ├── package.json
    ├── server.js
    ├── routes/
    ├── controllers/
    ├── services/
    ├── models/
    └── data/
```

5.2 API REST

A aplicação deverá disponibilizar uma API para a entidade principal do projeto.

A API deverá possuir, no mínimo:

- GET /api/entidade — listar registros;
- GET /api/entidade/:id — consultar um registro;
- POST /api/entidade — criar um registro;



- PUT ou PATCH /api/entidade/:id — alterar um registro;
- DELETE /api/entidade/:id — excluir um registro.

O nome entidade deverá ser substituído de acordo com o tema, por exemplo /api/livros, /api/eventos ou /api/produtos.

5.3 CRUD

O sistema deverá implementar:

- Create: criar registros;
- Read: consultar registros;
- Update: alterar registros;
- Delete: excluir registros.

Cada operação deverá possuir uma rota, retornar uma resposta adequada, informar erros quando necessário e utilizar códigos de status HTTP coerentes.

Exemplos:

- 200: operação realizada com sucesso;
- 201: registro criado;
- 400: dados inválidos;
- 404: registro não encontrado;
- 500: erro interno.

5.4 Persistência de dados

A aplicação deverá armazenar os dados em algum mecanismo persistente.

O grupo poderá utilizar:

- Banco de dados real, como MongoDB;

O projeto deverá explicar:

- Qual mecanismo foi utilizado;
- Como criar ou configurar o banco;
- Onde estão armazenados os dados;
- Como a aplicação realiza a conexão;



- Como executar o projeto em outro computador.

5.5 Integração entre front-end e back-end

O front-end deverá:

- Buscar dados da API;
- Enviar dados para a API;
- Atualizar registros por meio da API;
- Remover registros por meio da API;
- Atualizar a interface após as respostas do servidor.

5.6 Consumo de APIs

O grupo deverá utilizar:

- fetch;
- Promises;
- async;
- await;
- Tratamento de erros com try/catch.

5.7 Página dinâmica

A página deverá ser atualizada com base nos dados recebidos do serviço web. Deverá:

- Carregar registros ao abrir;
- Apresentar mensagens enquanto aguarda uma resposta;
- Informar quando não existem registros;
- Exibir mensagens de erro;
- Atualizar a lista após criar, alterar ou excluir um registro;
- Permitir filtros ou busca, quando aplicável ao tema.

5.8 Microserviços e rotas

O grupo deverá explicar na documentação como organizou o back-end.

Quando aplicável, poderá separar funcionalidades em serviços, como:



- Serviço de usuários;
- Serviço de produtos;
- Serviço de reservas;
- Serviço de autenticação;
- Serviço de notificações.

Não é obrigatório criar vários servidores independentes. É obrigatório demonstrar compreensão da separação entre rotas, controladores, serviços e modelos.

5.9 Documentação da API

A entrega deverá conter:

- Nome da API;
- Descrição;
- Endpoints;
- Métodos HTTP;
- Parâmetros;
- Corpo esperado;
- Respostas possíveis;
- Exemplos de requisição;
- Exemplos de resposta;
- Instruções de execução.

A documentação poderá ser escrita em Markdown.

5.10 Entrega da Parte 2

A entrega final deverá conter:

```
projeto_nome_grupo/  
├── README.md  
├── planejamento.pdf  
├── documentacao-api.md  
├── frontend/  
│   ├── index.html  
│   ├── css/  
│   │   └── style.css  
│   └── js/
```



```
|   └─ main.js
└─ backend/
    ├── package.json
    ├── server.js
    ├── routes/
    ├── controllers/
    ├── services/
    ├── models/
    ├── middleware/
    └─ data/
```

Se for utilizado banco de dados, o projeto deverá conter instruções de configuração, arquivo de exemplo com variáveis de ambiente, scripts de criação ou inicialização e orientações para restaurar os dados, quando necessário.

Não deverá ser enviado arquivo contendo senhas, tokens ou credenciais reais.

5.11 Avaliação da Parte 2

- Evolução coerente da Parte 1: 0,5 ponto;
- Organização do back-end: 1,0 ponto;
- Criação da API REST: 1,0 ponto;
- Implementação do CRUD: 1,5 ponto;
- Persistência de dados: 1,0 ponto;
- Rotas, controladores e serviços: 0,75 ponto;
- Integração entre front-end e API: 1,0 ponto;
- Uso de fetch, Promises e async/await: 0,75 ponto;
- Tratamento de erros e respostas HTTP: 0,5 ponto;
- Página dinâmica e usabilidade: 0,5 ponto;
- Documentação técnica: 0,5 ponto;
- Apresentação final: 1,0 ponto.

Totalizando 10 pontos, sujeito ao peso definido para a Parte 2 na avaliação da disciplina (disponível em plano de Ensino no moodle).

6. Cronograma sugerido

- Até 01/set: definição do tema, público e problema;
- Até 07/set estrutura HTML, CSS e formulário;



- Até 09/set: JavaScript, DOM, validação, eventos e módulos;
- 13/set: conclusão da Parte 1;
- 14/set: apresentação da Parte 1;
- 21/set a 30/set: Node.js, Express e API inicial;
- 05/out a 07/out: rotas, serviços e integração com armazenamento;
- 19/out a 28/out: banco de dados, fetch, Promises e async/await;
- 09/nov a 16/nov: integração, testes e finalização;
- 23/nov e 25/nov: apresentação final dos projetos.

7. Regras de autoria e apresentação

- O projeto deverá ser criado pelos próprios alunos.
- O uso de bibliotecas deverá ser informado.
- O uso de códigos externos deverá ser identificado na documentação.
- Cada integrante deverá compreender o código apresentado.
- O professor poderá solicitar execuções de CRUD durante a apresentação.
- A ausência de compreensão sobre partes fundamentais do código poderá afetar a avaliação individual.
- O grupo deverá demonstrar o funcionamento das rotas da API.
- O grupo deverá explicar o fluxo completo: usuário, front-end, requisição, API, serviço, banco ou armazenamento, resposta e atualização da interface.

8. Resultado esperado

Ao final do projeto, o grupo deverá apresentar uma aplicação web iniciada do zero, com uma primeira versão baseada em HTML, CSS e JavaScript e uma versão final integrada a um back-end, API REST e mecanismo de persistência.

A Parte 1 comprovará o domínio dos fundamentos de desenvolvimento no navegador. A Parte 2 demonstrará a evolução para uma aplicação cliente-servidor completa, acompanhando o plano de aula da disciplina até novembro.